

CodeNova Business Website

Technical Requirements Document (TRD)

Version: 1.0 **System:** CodeNova Agency Website + Lead Management Platform

1. Technical Architecture

- Recommended architecture: modern full-stack web application with a public marketing website, API/server layer, database and protected admin dashboard.
- Frontend: Next.js + TypeScript + Tailwind CSS.
- Server/API: Next.js server functions/API routes or a dedicated Node.js/Express backend.
- Database: PostgreSQL through Supabase.
- Authentication: Supabase Auth or secure server-side authentication.
- Deployment: Vercel for the web application and managed database through Supabase.
- External integrations should be isolated behind server-side API routes so secret keys are never exposed to browsers.

2. High-Level System Flow

- Visitor → Next.js Frontend → Server/API → PostgreSQL/Supabase.
- Contact/Estimator Form → Validation → API → Leads Database → Admin Dashboard.
- Consultation Request → Validation → Database → Email Notification / Calendar Integration.
- WhatsApp CTA → WhatsApp click-to-chat.
- Blog/Portfolio CMS → Admin Authentication → API → Database → Public SEO pages.
- Analytics → Google Analytics / Tag Manager → Conversion events.

3. Frontend Technical Requirements

- Use reusable React/Next.js components for Navbar, Footer, Hero, ServiceCard, PortfolioCard, CaseStudySection, CTA, FAQ, ContactForm and Estimator.
- Use responsive Tailwind breakpoints for mobile, tablet and desktop.
- Use semantic HTML and accessible form labels.
- Implement loading, empty, success and error states for dynamic content and forms.
- Use optimized responsive images and lazy loading where appropriate.
- Use server rendering/static generation for SEO-critical public pages where practical.

4. Recommended Route Structure

- /
- /services
- /services/web-development
- /services/web-applications
- /services/mobile-app-development

- /services/desktop-application-development
- /services/ai-agents
- /services/ai-automation
- /services/ecommerce-development
- /services/custom-software
- /services/ui-ux-design
- /solutions
- /portfolio
- /portfolio/[slug]
- /about
- /process
- /pricing
- /blog
- /blog/[slug]
- /project-estimator
- /book-consultation
- /contact
- /privacy-policy
- /terms-and-conditions
- /cookie-policy
- /refund-policy
- /admin
- /admin/leads
- /admin/services
- /admin/portfolio
- /admin/blog
- /admin/settings

5. Desktop Application Development — Technical Scope

- Create a dedicated route: /services/desktop-application-development.
- Support requirements for Windows, macOS and cross-platform applications.
- Potential technologies: Electron, Tauri, .NET/WPF, Java or another suitable technology based on project requirements.
- Potential capabilities: local database, offline mode, synchronization, printing, file management, POS, billing, inventory, dashboards, authentication, device integration and API connectivity.
- Desktop projects should use secure credential storage, signed builds where applicable, controlled update mechanisms and appropriate local-data protection.
- The website should present platform and technology choices as project-dependent options, not universal guarantees.

6. Backend / API Requirements

- Use REST-style JSON APIs or Next.js route handlers.
- Validate every client-supplied field on the server.
- Return consistent HTTP status codes and structured error responses.
- Use environment variables for secrets and service credentials.
- Add rate limiting to public forms and sensitive endpoints.
- Log server-side errors without exposing secrets or sensitive user data.

7. Core API Endpoints

- POST /api/leads — create a lead.
- GET /api/leads — admin-only lead listing.
- GET /api/leads/:id — admin-only lead details.
- PATCH /api/leads/:id — update lead status/notes.
- DELETE /api/leads/:id — delete a lead where permitted.
- POST /api/consultations — create consultation request.
- POST /api/estimator — calculate/store an indicative estimate.
- GET /api/services — public service data.
- GET /api/portfolio — public portfolio data.
- GET /api/blog — public blog listing.
- POST /api/admin/blog — admin-only blog creation.
- PATCH /api/admin/blog/:id — admin-only blog update.

8. Database Design

- users: id, name, email, auth reference, role, created_at, updated_at.
- leads: id, name, email, phone, company, country, service, budget, message, source, status, notes, assigned_to, created_at, updated_at.
- consultations: id, lead_id, preferred_date, preferred_time, status, meeting_url, notes, created_at.
- estimator_requests: id, lead_id, project_type, complexity, selected_features, estimated_min, estimated_max, created_at.
- services: id, slug, name, description, icon, status, sort_order, created_at, updated_at.
- portfolio_projects: id, slug, name, category, description, technologies, cover_image, content, status, created_at, updated_at.
- blog_posts: id, slug, title, excerpt, content, cover_image, category, author, seo_title, seo_description, published_at, status.
- testimonials: id, client_name, company, role, content, rating, image, status.
- site_settings: id, key, value, updated_at.

9. Database Relationships

- One lead can have multiple consultation records and estimator requests.

- Admin users can manage leads, services, portfolio and blog content.
- Portfolio and blog records use unique slugs for public URLs.
- Use indexes on email, status, service, created_at and slug fields where appropriate.

10. Authentication & Authorization

- Public visitors do not need accounts to browse the site or submit a basic inquiry.
- Admin routes require authenticated users.
- Use role-based authorization for admin operations.
- Verify authorization server-side; never trust role values supplied by the browser.
- Use secure session handling and appropriate expiration/refresh mechanisms.
- Add MFA for administrators when supported.

11. Lead Management Flow

- User submits contact form → client validation → server validation/sanitization → lead stored → notification sent → admin reviews → status/notes updated → follow-up.
- Lead statuses: New, Contacted, Qualified, Proposal Sent, Negotiation, Won and Lost.

12. Project Estimator

- Inputs: project type, complexity and selected features.
- Project types include Website, Web App, Mobile App, Desktop App, AI Agent, AI Automation, E-Commerce and Custom Software.
- Maintain pricing rules on the server or in an admin-managed configuration.
- Calculate an indicative minimum and maximum range.
- Store estimator submissions for analytics and follow-up when contact information is provided.
- Display that the estimate is indicative and final pricing depends on scope.

13. Admin Dashboard

- Overview: total leads, new leads, qualified leads, consultation requests and active projects.
- Lead table with search, filters, sorting, status management and lead details.
- Portfolio management: create, edit, publish/unpublish and delete.
- Blog management: draft, publish, edit, delete and SEO metadata.
- Service management: descriptions, ordering and visibility.
- Settings: business contact details, social links, CTA defaults and configurable site data.

14. Email & Notifications

- Use Resend, Brevo or another transactional email provider.
- Send visitor confirmation after successful inquiry when configured.
- Send internal notification to the CodeNova business inbox.
- Consultation workflows can send booking confirmation and reminders after calendar integration.

- Keep email templates separate from application logic where practical.

15. WhatsApp Integration

- Use a WhatsApp click-to-chat URL for the floating CTA.
- Prefill a short project inquiry message.
- Do not collect WhatsApp credentials in the website.

16. SEO Technical Implementation

- Unique metadata for every indexable page.
- Canonical URLs.
- Sitemap.xml and robots.txt.
- Open Graph metadata.
- Appropriate JSON-LD such as Organization, WebSite, Service, Article and LocalBusiness only when accurate.
- Clean semantic URLs and descriptive slugs.

17. Analytics & Tracking

- Google Analytics, Google Search Console and optionally Google Tag Manager.
- Track lead_submit, consultation_request, estimator_submit, whatsapp_click, phone_click and primary_cta_click.
- Track campaign/source data where possible.
- Do not send unnecessary personally identifiable information to analytics systems.

18. Security Requirements

- HTTPS in production.
- Secure HTTP headers where appropriate.
- Server-side input validation and sanitization.
- Parameterized queries/ORM to prevent SQL injection.
- Safe output rendering to reduce XSS risk.
- Rate limiting and CAPTCHA/anti-bot protection for public forms.
- All private keys and database credentials must remain server-side.
- Least-privilege database access policies.

19. Performance Requirements

- Image optimization and modern formats where appropriate.
- Lazy-load non-critical media.
- Minimize client-side JavaScript.
- Use caching for suitable public content.
- Target strong Core Web Vitals and Lighthouse performance.
- Monitor real-world performance after launch.

20. Accessibility Requirements

- Semantic HTML and keyboard navigation.
- Accessible form labels and visible focus states.
- Sufficient color contrast.
- Meaningful alt text.
- Do not communicate status using color alone.
- Support reduced-motion preferences.

21. Third-Party Integrations

- Supabase — database/authentication.
- Vercel — deployment.
- Google Analytics/Search Console — analytics and SEO.
- Resend/Brevo — transactional email.
- Google Calendar/Calendly — optional consultation scheduling.
- WhatsApp — direct client communication.
- Cloud image storage/CDN — optional.

22. Environment Variables

- NEXT_PUBLIC_SITE_URL
- NEXT_PUBLIC_ANALYTICS_ID
- SUPABASE_URL
- SUPABASE_ANON_KEY
- SUPABASE_SERVICE_ROLE_KEY — server only
- DATABASE_URL — if directly connecting to PostgreSQL
- EMAIL_PROVIDER_API_KEY — server only
- CALENDAR credentials — server only when enabled.
- Production secrets must be managed using secure hosting environment settings.

23. Testing Requirements

- Unit tests for estimator and utility functions.
- API tests for validation, authorization and lead creation.
- End-to-end tests for contact form, estimator, consultation request and admin login.
- Responsive testing on mobile, tablet and desktop.
- Cross-browser testing on current Chrome, Edge, Firefox and Safari where applicable.
- Security checks for authentication, input validation and public endpoints.
- SEO checks for metadata, sitemap, robots.txt, canonical URLs and broken links.

24. Deployment & CI/CD

- Use Git for source control.
- Recommended folders: app, components, lib, API routes, types, public, tests and database migrations.
- Maintain separate development and production environments.
- Run linting, type checking and automated tests before deployment.
- Use preview deployments for pull requests where supported.
- Use database migrations for schema changes.
- Keep production credentials separate from development credentials.

25. Monitoring & Maintenance

- Monitor application errors, failed API requests and uptime.
- Review database performance and storage.
- Update dependencies and address security advisories.
- Maintain backups according to provider and business requirements.
- Review analytics and conversion performance after launch.

26. Non-Functional Requirements

- Scalability: allow additional services, landing pages and admin modules.
- Reliability: maintain availability during expected traffic.
- Maintainability: reusable components, typed interfaces and clear module boundaries.
- Security: least privilege and secure secret handling.
- Usability: clear navigation and strong CTA hierarchy.
- SEO: public service and content pages should be crawlable when intended.

27. Recommended Project Structure

- app/ — routes and layouts
- components/ — reusable UI
- lib/ — database, auth, validation, email and utilities
- app/api/ — server/API routes
- types/ — TypeScript types
- public/ — static assets
- tests/ — unit, integration and end-to-end tests
- supabase/ or db/ — migrations and database configuration

28. Production Acceptance Criteria

- All primary pages work on supported screen sizes.
- Contact, estimator and consultation forms validate and submit successfully.
- Leads are stored correctly and visible only to authorized administrators.
- Admin routes are protected.

- Desktop Application Development appears in service navigation, services, estimator and relevant SEO content.
- SEO metadata, sitemap and robots configuration are valid.
- Analytics conversion events work correctly.
- No secret keys are exposed to the browser.
- Production build passes type checking, linting and automated tests.

Technical Summary

CodeNova should be implemented as a secure, responsive, SEO-friendly full-stack agency platform with a high-conversion public website and an extensible admin/lead-management system.

The PRD defines what the website should achieve. This TRD defines the recommended technical architecture, routes, APIs, database model, security, testing, deployment and operational requirements.